Target Tracking

This tutorial is an advanced version of the color block positioning experiment, adding robotic arm following functionality.

Overview: Target tracking includes three modes: color tracking, color picking tracking (learning tracking), and face tracking. The principle is to process images through the camera, identify targets through specific methods, obtain the target's coordinate position under the camera, calculate the deviation value between the target center point and the image center point, debug through PID algorithm, and drive the robotic arm to move. This makes the target center point coincide with the image center point.

- Start Docker

After entering the main board desktop, open the terminal and run the following command to start the corresponding Dofbot container:

```
./Docker_Ros.sh
```

Then, in the Docker terminal, input the following command to start jupyter-lab,

```
cd
jupyter-lab --allow-root
```

Then, in the browser, input the main board's IP address plus :9999,

```
IP:9999         // 示例：192.168.1.11:9999
```

```
Game Instructions:
1) First close the main program
2) Color tracking and color picking tracking (learning tracking) paths:
/root/Dofbot_2D/src/dofbot_color_follow/Color_tracking.ipynb
```

1. Control Design

- Import header files

```
import cv2 as cv
import threading
import random
from time import sleep
import ipywidgets as widgets
from IPython.display import display
from color_follow import color_follow
```

- Initialize robotic arm position

```
import Arm_Lib
Arm = Arm_Lib.Arm_Device()
joints_0 = [90, 135, 20, 25, 90, 30]
Arm.Arm_serial_servo_write6_array(joints_0, 1000)
```

- Create instance, initialize parameters

```
follow = color_follow()
# Initialize mode
model = 'General'
# Initialize HSV_learning value
HSV_learning = ()
# Initialize HSV values
color_hsv = {"red" : ((0, 43, 46), (10, 255, 255)),
        "green" : ((35, 43, 46), (77, 255, 255)),
        "blue" : ((100, 43, 46), (124, 255, 255)),
        "yellow": ((26, 43, 46), (34, 255, 255))}
# Set random colors
color = [[random.randint(0, 255) for _ in range(3)] for _ in range(255)]
```

- Create controls

```
button_layout = widgets.Layout(width='200px',
height='100px',align_self='center')
# Output widget
output = widgets.Output()
# Color tracking
color_follow = widgets.Button(description='color_follow', button_style='success',
layout=button_layout)
# Select color
choose_color = widgets.ToggleButtons(options=['red', 'green', 'blue', 'yellow'],
button_style='success',tooltips=['Description of slow', 'Description of regular',
'Description of fast'])
# Cancel tracking
follow_cancel = widgets.Button(description='follow_cancel',
button_style='danger', layout=button_layout)
# Learn color
learning_color = widgets.Button(description='learning_color',
button_style='primary', layout=button_layout)
# Learn color tracking
learning_follow = widgets.Button(description='learning_follow',
button_style='success', layout=button_layout)
# Exit
exit_button = widgets.Button(description='Exit', button_style='danger',
layout=button_layout)
# Image widget
imgbox = widgets.Image(format='jpg', height=480, width=640,
layout=widgets.Layout(align_self='auto'))
# Vertical layout
img_box = widgets.VBox([imgbox, choose_color],
layout=widgets.Layout(align_self='auto'))
# Vertical layout
Slider_box = widgets.VBox([color_follow, learning_color,
learning_follow,follow_cancel,exit_button],layout=widgets.Layout(align_self='aut
o'))
# Horizontal layout
controls_box = widgets.HBox([img_box, Slider_box],
layout=widgets.Layout(align_self='auto'))
# ['auto', 'flex-start', 'flex-end', 'center', 'baseline', 'stretch', 'inherit',
'initial', 'unset']
```

- Mode switching

```
def color_follow_Callback(value):
      global model
      model = 'color_follow'

def learning_color_Callback(value):
      global model
      model = 'learning_color'

def learning_follow_Callback(value):
      global model
      model = 'learning_follow'

def follow_cancel_Callback(value):
      global model
      model = 'General'

def exit_button_Callback(value):
      global model
      model = 'Exit'
color_follow.on_click(color_follow_Callback)
learning_color.on_click(learning_color_Callback)
learning_follow.on_click(learning_follow_Callback)
follow_cancel.on_click(follow_cancel_Callback)
exit_button.on_click(exit_button_Callback)
```

- Main program

```
def camera():
    global HSV_learning,model
    # Open camera
    capture = cv.VideoCapture(0)
    capture.set(3, 640)
    capture.set(4, 480)
    capture.set(5, 30)  # Set frame rate
    # Be executed in loop when the camera is opened normally
    while capture.isOpened():
        try:
            # Read each frame from camera
            _, img = capture.read()
            # Unify image size
            img = cv.resize(img, (640, 480))
            if model == 'color_follow':
                img = follow.follow_function(img,color_hsv[choose_color.value])
                # Add text

                cv.putText(img, choose_color.value, (int(img.shape[0] / 2), 50),
cv.FONT_HERSHEY_SIMPLEX, 2, color[random.randint(0, 254)], 2)
            if model == 'learning_color':
                img,HSV_learning = follow.get_hsv(img)
            if model == 'learning_follow':
                img = follow.learning_follow(img, HSV_learning)
                # Add text
                cv.putText(img,'LeColor', (200, 50), cv.FONT_HERSHEY_SIMPLEX, 1,
color[random.randint(0, 254)], 1)
            if model == 'Exit':
```

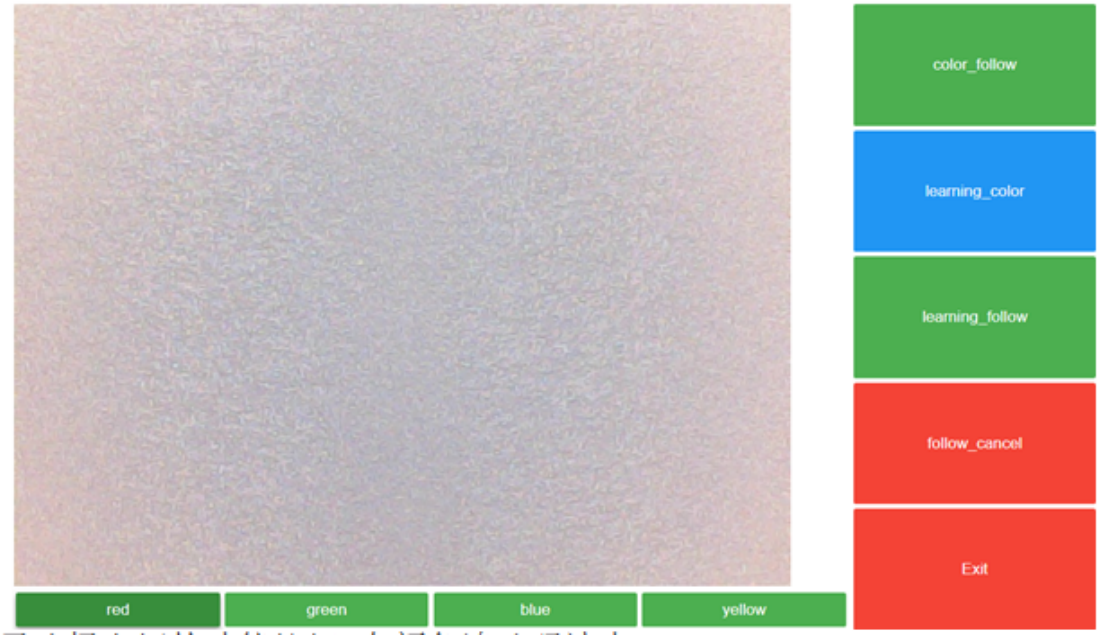
```
                cv.destroyAllWindows()
                capture.release()
               break

            imgbox.value = cv.imencode('.jpg', img)[1].tobytes()
        except KeyboardInterrupt:capture.release()
```

- Start

```
display(controls_box,output)
threading.Thread(target=camera, ).start()
```

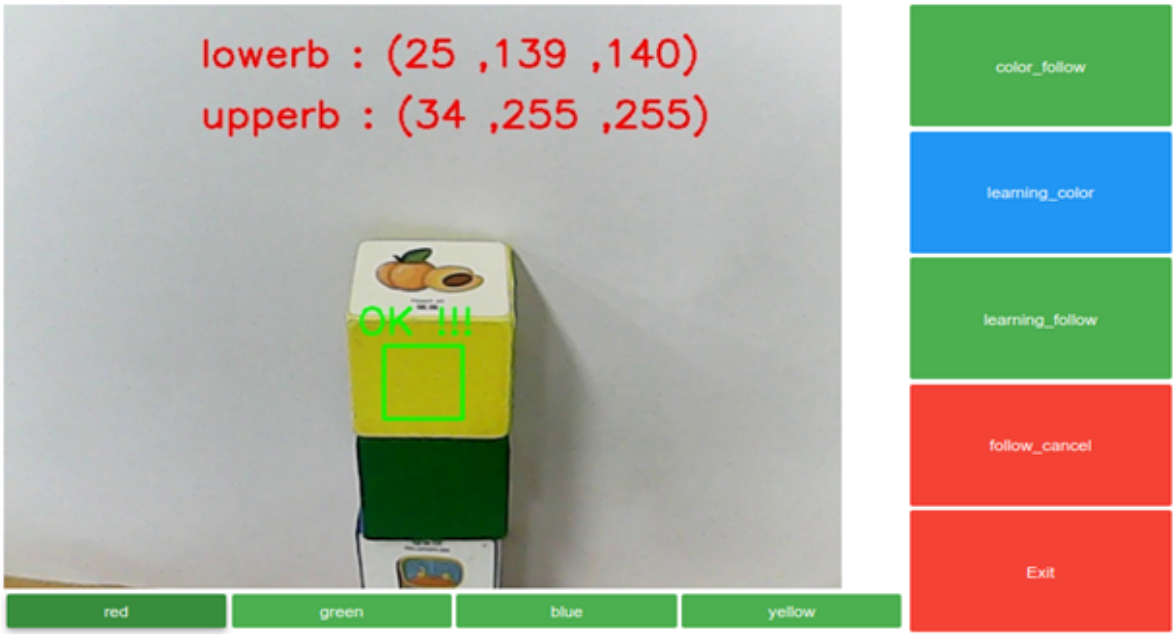
```
    The adjustment of the control algorithm PID is explained in the PID algorithm
section. If adjustment is needed, please refer to the PID algorithm for learning.
The library code for color tracking and color picking tracking can be found at
/root/Dofbot_2D/src/dofbot_color_follow/color_follow.py
```

2. Color Tracking




```
    By default, the game does not track. You need to select the function button
on the right side. In the color tracking mode,

    Start the code block, click the [color_follow] button to enable color
tracking. You can switch colors below the image. Place the block in the field of
view, wait for the camera to recognize it, and the robotic arm will track the
block's movement, facing the center of the block.

    Click the [follow_cancel] button to cancel tracking without exiting the
program. Click the Exit button to exit the program.
```

3. Color Picking Tracking



(1) Start the code block, click the [learning_color] button, a square box appears in the center of the screen, as shown in the left image.

(2) Place an object within the square box, the HSV high and low thresholds are printed in real-time. When [OK !!!] appears below the square box, it indicates successful recognition.

(3) Click the [learning_follow] button, as long as the learned color contour can be detected, it can track in real-time.

(4) Click the [follow_cancel] button to cancel tracking without exiting the program. Click the Exit button to exit the program.

4. Code Analysis

- Get HSV range of a specific area

```
# Convert color image to HSV
HSV = cv.cvtColor(img, cv.COLOR_BGR2HSV)
# Draw rectangle
cv.rectangle(img, (290, 190), (350, 250),   (0, 255, 0), 2)
# Take out H, S, V values of each row and column and put them into containers
for i in range(290, 350):
      for j in range(190, 250):
          H.append(HSV[j, i][0])
          S.append(HSV[j, i][1])
          V.append(HSV[j, i][2])
```

- Calculate H, S, V maximum and minimum values respectively

```
H_min = min(H);H_max = max(H)
S_min = min(S);S_max = max(S)
V_min = min(V);V_max = max(V)
```

- HSV range adjustment, HSV values of blocks at different positions in the image have certain differences, so appropriate adjustment is needed.

```
if H_max + 2 > 255:H_max = 255
else:H_max += 2
if H_min - 2 < 0:H_min = 0
else:H_min -= 2
if S_min-10<0:S_min=0
else:S_min -= 15
if V_min-10<0:V_min=0
else:V_min -= 15
S_max = 255; V_max = 255
```

- Format conversion, convert calculated data to int format

```
hsv_range = ((int(H_min), int(S_min),   int(V_min)),(int(H_max), int(S_max),
int(V_max)))
```

The subsequent contour detection, PID control and tracking are the same as color tracking.